# CS 5150: Software Engineering

Prof. Saikat Dutta

Spring 2026

www.cs.cornell.edu/courses/cs5150

# The Pitch

- You already know how to write **programs**
- But can you create a **software system**?
  - Example: Canvas, CMSX, Google?
- If I offered to pay you to automate some work, could you answer these basic questions?
  - How long will it take? How many developers will you need?
  - How will you know when you're done?
  - How often will it crash?
  - Is it even legal?

# About the course

**Objective**: Prepare you to contribute to reliable software systems

- Semester-long **team project**
- 4 credits = 10+ hours of work per week (outside of lecture)
  - Regular team meetings instead of discussion section
- 5000-level: assume you can **teach yourself** new technical skills
- Focus on expanding capabilities of **large, existing** codebases
- **PreReqs**: Strong programming experience! CS 2110/3110/4410, etc.

# About your instructor: Prof. Saikat Dutta

- Completed PhD in 2023 in Computer Science (UIUC)
- Research Interests: Software Engineering x Machine Learning
- Joined Cornell in Fall 2024 as an Assistant Professor
- Industry exp: Microsoft, MSR, and AWS
- **Languages**: Basic, C/C++, Java, Python
- **Email**: saikatd@cornell.edu
- **Office hours**: Mon 1 PM – 2 PM Gates 438

# Course Staff

- Joonhwan Yoo, jy345@cornell.edu
- Stephen Shen, zs435@cornell.edu
- Junkai Huang, jh2973@cornell.edu
- Kabir Anirudh Samsi, kas499@cornell.edu
- Harsh Patel, hhp26@cornell.edu
- Kaityln Lu, kl995@cornell.edu
- **TA office hours**: Wed 1PM - 2 PM, Room: TBD

TA
Joonhwan Yoo

TA
Junkai Huang

TA
Stephen Shen

TA
Harsh Patel

TA
Kaitlyn Lu

TA
Kabir Anirudh Samsi

# Illustrated with successes and failures

- Linux kernel
  - Distributed development, review
- SQLite
  - Testing – 640x more test code
- Blender & Krita
  - User Collaboration, Open Develop.
- Dragon 2 spacecraft (SpaceX)
  - Agile Development

- Ariane 5 (1996)
- Boeing 737 MAX (2018,19)
- FAA Advanced Automation System
- Therac-25 (1985-87): radiation overdose
- …

# Software bugs…

CROWDSTRIKE — BUGS — CYBERSECURITY — NEWS

## CrowdStrike bug maxes out 100% of CPU, requires Windows reboots

"Note: This is 100% of a single core. In an 8-core system for example, an additional 12.5% of unexpected total CPU load would be experienced..."

THE STACK

June 28, 2024 . 4:18 PM — 2 min read

The Washington Post

*Democracy Dies in Darkness*

Technology

## Tesla sued by family of Apple engineer killed in Autopilot crash

"Tesla's Autopilot feature was defective and caused Huang's death," attorneys for Walter Huang said.

The New York Times

## *Airline Blames Bad Software in San Francisco Crash*

LOSSES FROM SOFTWARE FAILURES (USD)

TRICENTIS

**1,715,430,778,504**

ONETRILLIONSEVENHUNDREDFIFTEENBILLIONFOURHUNDREDTHIRTYMILLIONSEVENHUNDREDSEVENTYEIGHTTHOUSANDFIVEHUNDREDFOUR

# Activity: So, what is software engineering?

1. Turn to your neighbors. Introduce each other.
2. Discuss: what makes “software engineering” different from “programming”?
3. Select a reporter

# Textbook definitions

- “An engineering discipline that is concerned with all aspects of software production”
- “Multi-person development of a multi-version program”
- “Programming integrated over time”
- “When ‘clever’ is an accusation rather than a compliment”

# Software Engineering: Themes of this course

- Software Engineering involves the principles and practices for …
  - Software Specification (Requirements)
  - Software Design (Modeling)
  - Software Implementation (Programming)
  - Software Analysis (Performance)
  - Software Management  and Team work
  - Quality Assurance (Testing, Verification, Repair, …)
  - Software Maintenance
  - Professionalism
  - Delivery
  - …

# Syllabus

# Course Infrastructure

- Website: www.cs.cornell.edu/courses/cs5150/
  - Slides, assignments, schedule, syllabus
- Canvas (restricted to enrolled students)
  - Ed Discussion, Gradescope
  - Quizzes, surveys, etc.
- Poll Everywhere: pollev.com/cs5150sp26 (use Cornell e-mail)

# Example poll

PollEv.com/cs5150sp26

# Personal devices

- Laptops/tablets may be useful in class for:
  - Responding to polls
  - Following along with demos
  - Taking notes (but paper is better)
- BUT devices are distracting for you *and your neighbors*
- Please do not multitask during lecture. If you can't resist, sit in the back

Computers & Education 62 (2013) 24–31

Contents lists available at SciVerse ScienceDirect

Computers & Education

journal homepage: www.elsevier.com/locate/compedu

## Laptop multitasking hinders classroom learning for both users and nearby peers

Faria Sana [a], Tina Weston [b,c], Nicholas J. Cepeda [b,c,*]

[a] McMaster University, Department of Psychology, Neuroscience, & Behaviour, 1280 Main Street West, Hamilton, ON L8S 4K1, Canada
[b] York University, Department of Psychology, 4700 Keele Street, Toronto, ON M3J 1P3, Canada
[c] York University, LaMarsh Centre for Child and Youth Research, 4700 Keele Street, Toronto, ON M3J 1P3, Canada

ARTICLE INFO

*Article history:*
Received 11 September 2012
Received in revised form
5 October 2012
Accepted 12 October 2012

*Keywords:*
Laptops
Multitasking
Attentional control
Pedagogy

ABSTRACT

Laptops are commonplace in university classrooms. In light of cognitive psychology theory on costs associated with multitasking, we examined the effects of in-class laptop use on student learning in a simulated classroom. We found that participants who multitasked on a laptop during a lecture scored lower on a test compared to those who did not multitask, and participants who were in direct view of a multitasking peer scored lower on a test compared to those who were not. The results demonstrate that multitasking on a laptop poses a significant distraction to both users and fellow students and can be detrimental to comprehension of lecture content.

© 2012 Elsevier Ltd. Open access under CC BY-NC-ND license.

*"...participants who multitasked on a laptop during a lecture scored lower* ***on a test compared to those who did not multitask****, and participants who were in direct view of a multitasking peer scored lower on a test compared to those who were not. The results demonstrate that multitasking on a laptop poses a significant distraction to both users and fellow students and can be detrimental to comprehension of lecture content."*

# Assessment

- Lecture polls/In-class activities/Pop quizzes
  - Perfect attendance ***not*** necessary
- Canvas Assignments
  - 3-4 assignments
- Project deliverables
  - Reports, documentation, code, code reviews
  - Scored against rubric
  - Client presentations
  - Peer reviews
- In-class exams (2): Mar 10, Apr 16 (tentative)
- No Final Exam!

# Grading

- Project: 60%
- In-class exams (2): 20%
- Attendance/In-class activities: 10%
- Assignments: 10%

# References

- No required textbook
  - All readings freely available in electronic form
- Several recommended books, including a free one:
  - *Software Engineering at Google* https://abseil.io/resources/swe-book
- *Better Embedded System Software*. Philip Koopman
- *The Mythical Man-Month*. Frederick P. Brooks, Jr
- *Software Engineering, Tenth Edition*. Ian Sommerville

# Course is almost full!

If you decide this course isn't for you, that's okay. But please drop promptly to make room for students on the waitlist.

I cannot predict if you will get in!

For those on the waitlist (or waiting to get on the waitlist):

The instructor does not control enrollment; send questions to https://tdx.cornell.edu/TDClient/193/Portal/Home/
PINs will be sent automatically and regularly to students in the waitlist, based on available seats.

# Disclaimers about CS 5150

- This is a new version of 5150 – so schedule might change at any time! (but hopefully not by much) – please be patient!

- Please use **Ed** to ask questions – fastest way to get response from staff (reserve emails for personal/important things)
  - We will try to respond within 24 **business** hours

- TAs as project clients – please be respectful to them

# The Project

# Purpose of the project

- Experience interacting with a client other than yourself
  - Requirements elicitation, acceptance testing
- Experience designing for users other than yourself
- Experience all phases of software lifecycle
  - Specification, development, validation, evolution
- Practice coordinating with a team

# Teams

- Team size: 4-5 students
- Full-semester commitment
- Two options for forming teams (by Jan 29):
  1. Form a (partial or complete) team around a project
  2. Get matched with a team based on preferences
- When considering teammates, check for:
  - Compatible schedules, work styles
  - Backend language comfort (Java, Python)
  - 2+ members with frontend experience (HTML/CSS/JS)
- More about working in teams later

# Project options

**Option 1 (Internal)**

- Course-sponsored project (e.g., enhancing code review tools)
  - Print a review log
  - Link comments to multiple files
  - …
- Instructor or TA will serve "client" role
- You will propose features you want to implement

**Option 2 (External)**

- You find a client with a project proposal
  - Client may not be yourselves or other students
  - Must involve an existing, active codebase (not written by you)
- Complete survey: "petition for external project"
- Recruit on Ed Discussions
- We will advertise some external clients. See "external-clients.txt" under Modules/Project.

# Project Timeline

- Semester consists of five 3-week “sessions”
- Report due at the end of each session
- Should meet with client once per session
- Two formal presentations: **Midpoint** and **Final**
- Final delivery on the last day of classes

- **Time** and **resources** are fixed, so must adjust **scope** to fit in time available

# Project Timeline

| **Weeks** | **Tasks** |
|---|---|
| Week 1-3 | Choose Project and Form Teams |
| Weeks 4-6 | Sprint 1 |
| Weeks 7-10 | Sprint 2 |
| Weeks 7-9 | Midpoint Presentation |
| Weeks 11-14 | Sprint 3 |
| Weeks 15-16 | Sprint 4 |
| Weeks 15-16 | Final Presentation |

# Deliverables

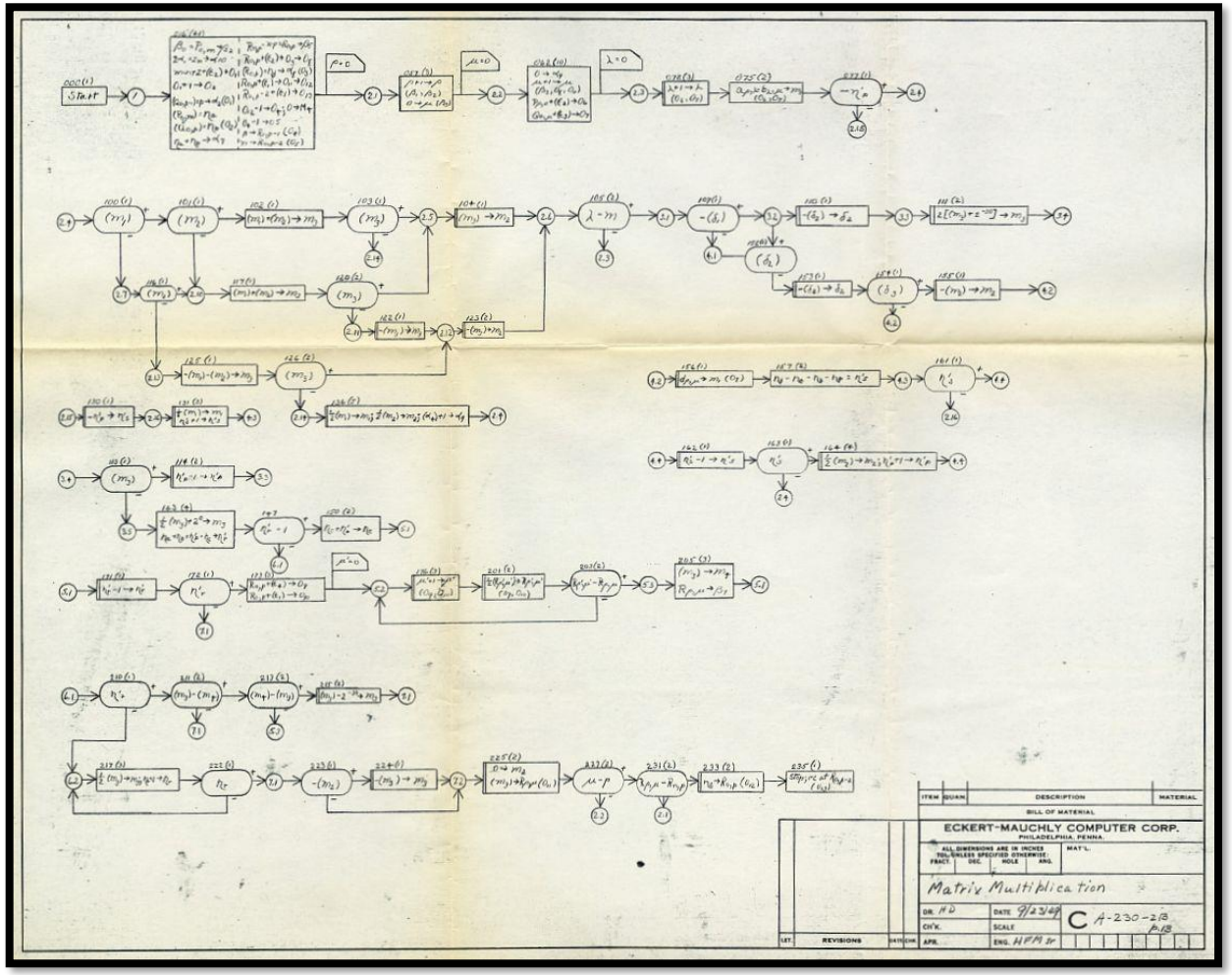
- Session reports
  - Plan
  - Accomplishments, setbacks, discovered work
  - Peer feedback
- Design documentation
- Test plans and reports
- Coverage analysis
- Requirements
- Code and code reviews
- User documentation

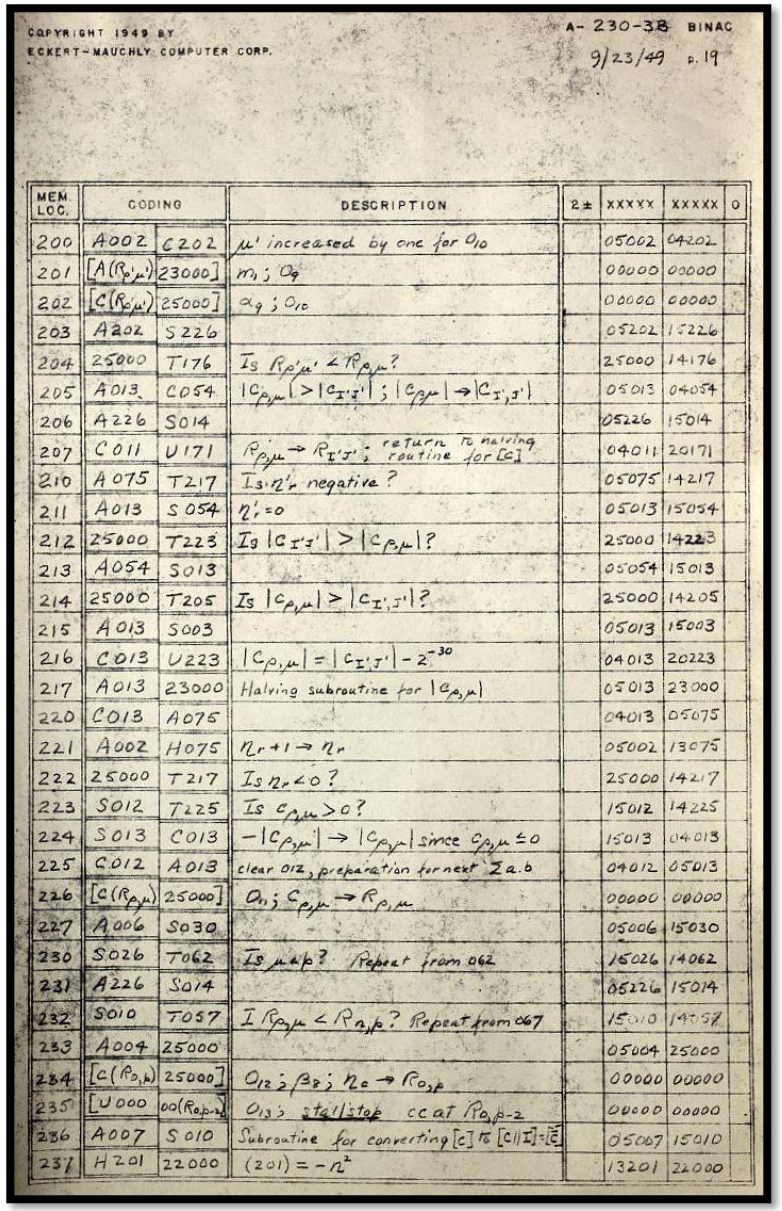
# Project Pitch

- Select a project and propose a feature that you would like to implement (See canvas). **Deadline**: Jan 27 11.59 PM EST.
- Few pre-approved projects and features are available on course website
- Goal is to assign unique project/feature to each team and match students to appropriate teams and projects
- Start forming teams. You may submit the pitch yourself or as a team.
- Submit **team matching survey** if you are looking for a team!
  - https://forms.gle/6o5GRqXnpKhd4CQ88

# Some history

# Early software development



COPYRIGHT 1949 BY ECKERT-MAUCHLY COMPUTER CORP.

A- 230-3B BINAC 9/23/49 p. 19

| MEM. LOC. | CODING | | DESCRIPTION | 2± | XXXXX | XXXXX | O |
|---|---|---|---|---|---|---|---|
| 200 | A002 | C202 | $\mu'$ increased by one for $O_{10}$ | | 05002 | 04202 | |
| 201 | [A($R_{\rho'\mu'}$) | 23000] | $m_1$; $O_9$ | | 00000 | 00000 | |
| 202 | [C($R_{\rho'\mu'}$) | 25000] | $\alpha_g$; $O_{10}$ | | 00000 | 00000 | |
| 203 | A202 | S226 | | | 05202 | 15226 | |
| 204 | 25000 | T176 | Is $R_{\rho'\mu'} < R_{\rho\mu}$? | | 25000 | 14176 | |
| 205 | A013 | C054 | $\lvert c_{\rho\mu}\rvert > \lvert c_{I'J'}\rvert$; $\lvert c_{\rho\mu}\rvert \to \lvert c_{I'J'}\rvert$ | | 05013 | 04054 | |
| 206 | A226 | S014 | | | 05226 | 15014 | |
| 207 | C011 | U171 | $R_{\rho\mu} \to R_{I'J'}$; return to halving routine for [c] | | 04011 | 20171 | |
| 210 | A075 | T217 | Is $\eta'_r$ negative? | | 05075 | 14217 | |
| 211 | A013 | S054 | $\eta'_r = 0$ | | 05013 | 15054 | |
| 212 | 25000 | T223 | Is $\lvert c_{I'J'}\rvert > \lvert c_{\rho\mu}\rvert$? | | 25000 | 14223 | |
| 213 | A054 | S013 | | | 05054 | 15013 | |
| 214 | 25000 | T205 | Is $\lvert c_{\rho\mu}\rvert > \lvert c_{I'J'}\rvert$? | | 25000 | 14205 | |
| 215 | A013 | S003 | | | 05013 | 15003 | |
| 216 | C013 | U223 | $\lvert c_{\rho\mu}\rvert = \lvert c_{I'J'}\rvert - 2^{-30}$ | | 04013 | 20223 | |
| 217 | A013 | 23000 | Halving subroutine for $\lvert c_{\rho\mu}\rvert$ | | 05013 | 23000 | |
| 220 | C013 | A075 | | | 04013 | 05075 | |
| 221 | A002 | H075 | $\eta_r + 1 \to \eta_r$ | | 05002 | 13075 | |
| 222 | 25000 | T217 | Is $\eta_r < 0$? | | 25000 | 14217 | |
| 223 | S012 | T225 | Is $c_{\rho\mu} > 0$? | | 15012 | 14225 | |
| 224 | S013 | C013 | $-\lvert c_{\rho\mu}\rvert \to \lvert c_{\rho\mu}\rvert$ since $c_{\rho\mu} \le 0$ | | 15013 | 04013 | |
| 225 | C012 | A013 | clear 012, preparation for next $\Sigma a.b$ | | 04012 | 05013 | |
| 226 | [C($R_{\rho\mu}$) | 25000] | $O_{11}$; $c_{\rho\mu} \to R_{\rho\mu}$ | | 00000 | 00000 | |
| 227 | A006 | S030 | | | 05006 | 15030 | |
| 230 | S026 | T062 | Is $\mu < p$? Repeat from 062 | | 15026 | 14062 | |
| 231 | A226 | S014 | | | 05226 | 15014 | |
| 232 | S010 | T057 | I $R_{\rho\mu} < R_{n,p}$? Repeat from 067 | | 15010 | 14057 | |
| 233 | A004 | 25000 | | | 05004 | 25000 | |
| 234 | [C($R_{0,p}$) | 25000] | $O_{12}$; $\beta_8$; $\eta_c \to R_{0,p}$ | | 00000 | 00000 | |
| 235 | [U000 | 00($R_{0,p-2}$)] | $O_{13}$; stall/stop cc at $R_{0,p-2}$ | | 00000 | 00000 | |
| 236 | A007 | S010 | Subroutine for converting [c] to [c‖I]=[$\bar{c}$] | | 05007 | 15010 | |
| 237 | H201 | 22000 | (201) = $-n^2$ | | 13201 | 22000 | |



Grace Murray Hopper Collection, National Museum of American History

# Structured Programming

- Arbitrary flowcharts hard to understand, verify
  - Yield programs with many GOTO statements
  - “Spaghetti code”
- Can restrict yourself to common patterns:
  - Control structures
    - Sequence
    - Selection (if-then-else)
    - Iteration (for, while)
  - Subroutines (named sequence)
- 1968: Edsger Dijkstra publishes *Go To Statement Considered Harmful*

# Software Crisis

- 1960s: computers getting faster, can tackle bigger problems
- Software couldn’t keep up
  - Over budget
  - Behind schedule
  - Buggy
  - Unmaintainable

*The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.*

- Edsger Dijkstra, 1972

# NATO software engineering conferences

- Held in 1968, 1969 to address the “software crisis”
  - David Gries in attendance
- Admitted inadequacy of contemporary methods
- Agreed upon best practices for developing software, grounded in principles of engineering
- Established the term “software engineering”

# Incremental improvements

- Structured programming
- Information hiding
- Object-oriented programming
- Language enforcement (Ada)
- Formal methods (CS 4160)
- CASE tools
- UML
- Components & services
- Agile methods
- The Internet
- Open source
- Code forges

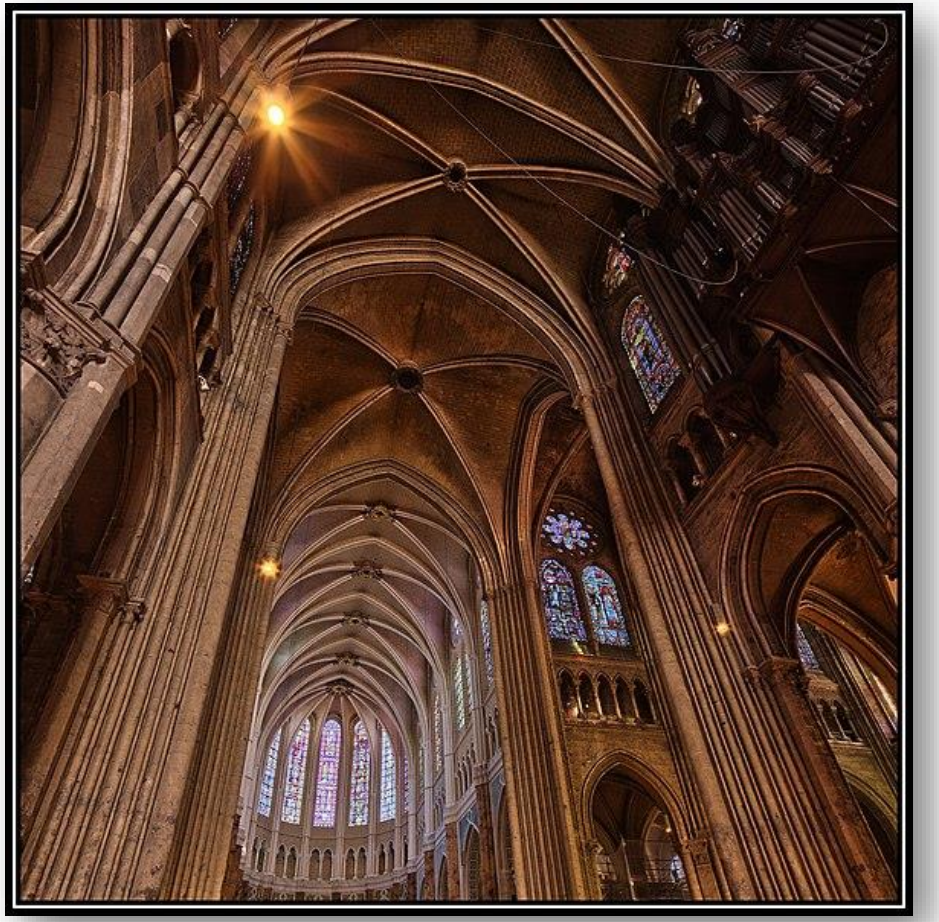
## Are we still in crisis?

- No new technology, tool, or process was a "silver bullet"
- Software has expanded in capability and scale, but …
  - Projects still behind schedule, over budget
  - Inefficiency has been masked by hardware advances
  - Much software is low-quality (e.g. security)
- And the stakes are higher than ever!
  - Software governs much of modern society

# Why not treat SW like HW?

Engineers have built large, reliable structures for millennia. Why not take same approach to SW?

- "Easy" to change
  - But costly in time, risk
- More coupled
- Leaner supply chain
- No unit production cost
- Scalable access
- Different reliability analysis




CC-By-SA Michel Mensler

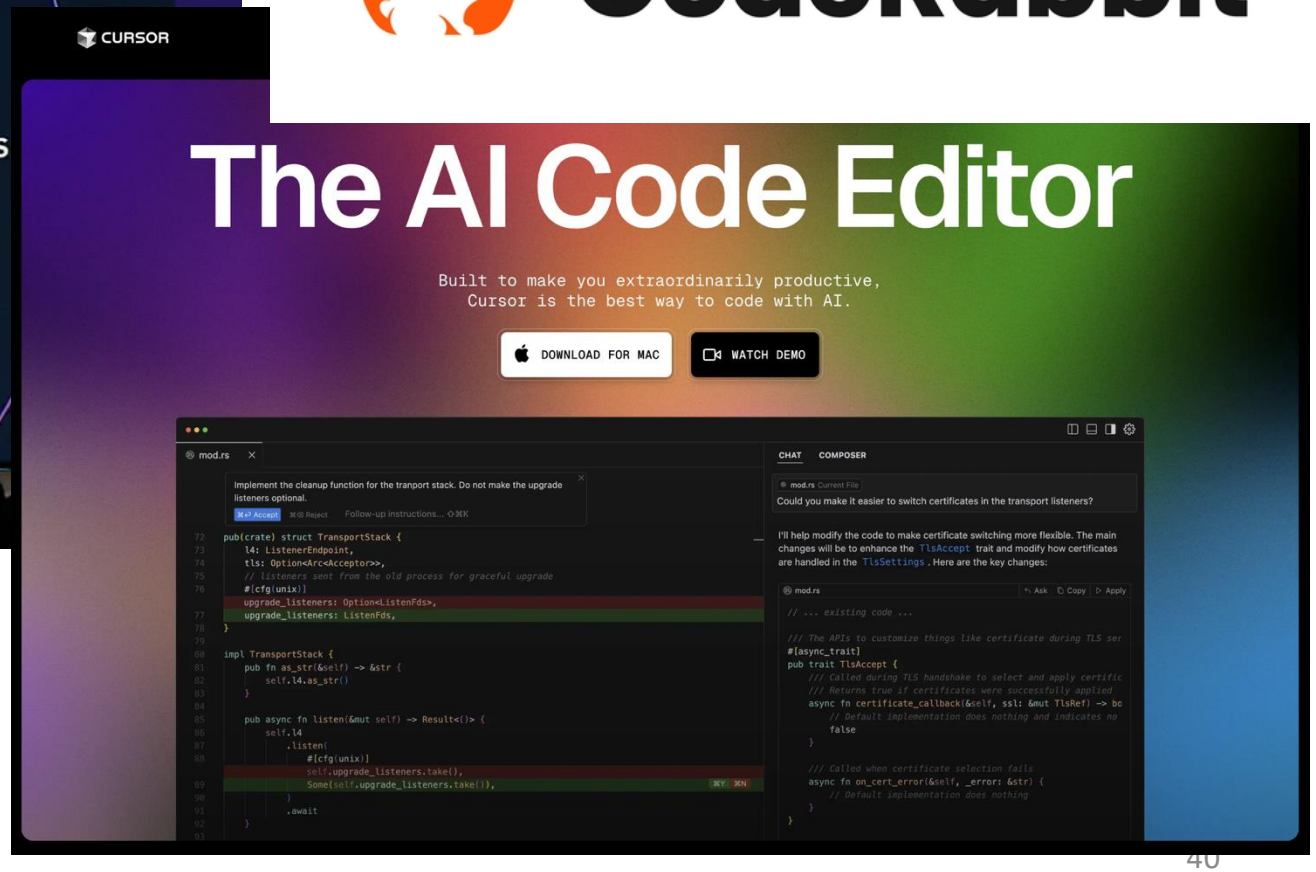
# Context of modern software engineering

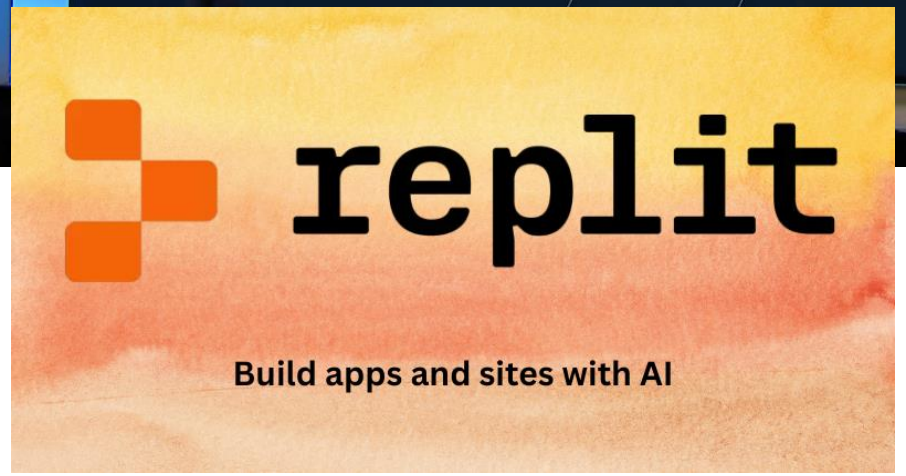
https://www.informationisbeautiful.net/visualizations/million-lines-of-code/

- Can you guess LOCs for Linux Kernel, FB, Google?
- Most software development is not "greenfield"
  - Far more time spent in ongoing development/maintenance than in initial development
  - Most software needs to interact closely with existing systems
  - Want to leverage reuse

# Software Engineering Today …

CodeRabbit



# Software Engineering Today …

**Andrej Karpathy**
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · **4.5M** Views

Search Dictionary

Slang & Trending

## vibe coding

*noun*

Writing computer code in a somewhat careless fashion, with AI assistance

### What does *vibe coding* mean?

*Vibe coding* (also written as *vibecoding*) is a recently-coined term for the practice of writing code, making web pages, or creating apps, by just telling an AI program what you want, and letting it create the product for you. In *vibe coding* the coder does not need to understand how or why the code works, and often will have to accept that a certain number of bugs and glitches will be present. The verb form of the word is *vibe code*.

# Software Engineering Today …

From same NYT article: *"Among college graduates ages 22 to 27, computer science and computer engineering majors are facing some of the* ***highest unemployment rates, 6.1 percent and 7.5 percent respectively****, according to a report from the Federal Reserve Bank of New York. That is more than double the unemployment rate among recent biology and art history graduates, which is just 3 percent."*

The New York Times

**Goodbye, $165,000 Tech Jobs. Student Coders Seek Work at Chipotle.**

As companies like Amazon and Microsoft lay off workers and embrace A.I. coding tools, computer science graduates say they're struggling to land tech jobs.

"The unfortunate thing right now, specifically for recent college grads, is those positions that are most likely to be automated are the **entry-level positions** that they would be seeking" – Matthew Martin, US economists

# Software Engineering Today …

- 92% of US-based developers report using GAI in- or outside work [1]
- 51% of professional developers report using GAI daily [4]
- At Microsoft, maybe 20–30% of code is written by GAI [2]
- A quarter of recent startups have code bases where ≥ 95% of code written by GAI [3]
- **But most developers (72%) are not vibe coding as of June 2025 [4]**


1. https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience
2. https://www.cnbc.com/2025/04/29/satya-nadella-says-as-much-as-30percent-of-microsoft-code-is-written-by-ai.html
3. https://techcrunch.com/2025/03/06/a-quarter-of-startups-in-ycs-current-cohort-have-codebases-that-are-almost-entirely-ai-generated
4. https://survey.stackoverflow.co/2025/ai

# AI Policy for this course

- You are free to use AI tools for coding; but please document usages in your reports!
- AI tools must not be used for in-class assignments, exams, pop quizzes, etc, unless otherwise stated!

# Next Up

1. Read about FAA AAS before next lecture!
2. Start forming teams around projects! Due: Jan 29
3. Work on project pitch! Due: Jan 27
4. Submit team matching survey! Due: Jan 27